

拓扑 AC CSP-S & NOIP 联测第二轮第三场讲评

袁胜利

2026 年 8 月 14 日

吐槽环节/自由分享

跳跃表

对于一个跳表，有 N 个有序元素，为元素 i 选择高度 $1 \leq h_i \leq K$ 。查询从最高层表头开始：能向右就向右，否则下降；只计算向右移动次数。元素 i 被查询 x_i 次，求所有查询的最小总代价。

$$N, K \leq 200, \quad 0 \leq x_i \leq 10^8.$$

跳跃表

子任务一 (10 分, $K = 1$): 所有元素都只在底层出现。查询 i 恰好从表头向右移动 i 次, 因此

$$\text{ans} = \sum_{i=1}^N ix_i.$$

时间复杂度为 $O(N)$, 额外空间复杂度为 $O(1)$ 。

观察

考虑 $K = 2$ 。设第二层最后一个节点为 p ，此前一共选了 c 个第二层节点。若下一个第二层节点为 q ，则：

- $p < j < q$ ：先在第二层移动 c 次，再在底层移动 $j - p$ 次；
- $j = q$ ：只在第二层移动 $c + 1$ 次。

新增贡献为

$$\sum_{j=p+1}^{q-1} (c + j - p)x_j + (c + 1)x_q.$$

预处理 $\sum x_j$ 与 $\sum jx_j$ 后可以 $O(1)$ 求出。

跳跃表

子任务二 (15 分, $K = 2$): 令 $dp[p][c]$ 表示已经确定到节点 p , 其中 p 是最后一个第二层节点、第二层共选了 c 个节点时的最小代价。

枚举下一个第二层节点 $q > p$, 加入上一页的整段贡献。停止选择后, 还要加入尾段

$$\sum_{j=p+1}^N (c + j - p)x_j.$$

状态数为 $O(N^2)$, 每个状态枚举 q , 时间复杂度为 $O(N^3)$, 空间复杂度为 $O(N^2)$ 。

跳跃表

子任务三 (15 分, $N \leq 15$ 且 $K \leq 3$): 每个高度 h_i 有 K 种选择, 直接枚举全部 K^N 个高度向量。

固定高度后, 对每个目标从第 K 层开始模拟查询; 若当前累计代价已经不小于已有最优值, 立即停止本次模拟。

直接实现的上界为 $O(K^N KN^2)$, 空间复杂度为 $O(N)$ 。

每次非空查询至少要向右移动一次，所以所有方案都有下界

$$\sum_{i=1}^N x_i.$$

若 $K \geq N$ ，可以令元素 i 的高度为 i (若 $K > N$ ，多出的高层留空)。查询元素 i 时，先下降到第 i 层，再从头结点一步走到 i ，所以下界可以达到。
若 $K = N - 1$ ，只能有一个编号 $i \geq 2$ 的查询多走一步；让 x_i 最小的元素承担这一步即可。编号 1 不能承担额外移动，因为它在底层也只需一步。

子任务四 (15 分, $K \geq N - 1$):

$$\text{ans} = \begin{cases} \sum_{i=1}^N x_i, & K \geq N, \\ \sum_{i=1}^N x_i + \min_{2 \leq i \leq N} x_i, & K = N - 1. \end{cases}$$

当 $N = 1$ 时必有 $K \geq N$, 不会进入第二种情况。时间复杂度为 $O(N)$ 。

加入编号为 0 的表头。令

$$F(l, r, h)$$

表示当前位于第 h 层节点 l ，所有目标位于 $(l, r]$ 时，安排区间内部节点高度所需的最小后续代价。

这个状态把问题分成两部分：当前层右侧第一个节点以前的查询只能下降；到达第一个节点以后的查询仍留在当前层。记

$$S_i = \sum_{j=1}^i x_j.$$

边界为

$$F(l, l, h) = 0,$$

$$F(l, r, 1) = \sum_{j=l+1}^r (j - l)x_j.$$

若 $h \geq r - l$, 可以用不同层依次落到每个目标, 使所有查询都只横移一次:

$$F(l, r, h) = S_r - S_l.$$

这也覆盖了 $K > N$ 的情况; 较高层允许为空。

跳跃表

先写不带剪枝的完整转移。第 h 层可以不再放节点，代价为 $F(l, r, h - 1)$ 。
否则枚举本层右侧第一个节点层数与 l 相同都为 h 层的点 i (可能不存在)。

- (l, i) 内的目标下降一层，代价 $F(l, i - 1, h - 1)$;
- 所有目标 $i, i + 1, \dots, r$ 都先从 l 横移到 i ，贡献 $S_r - S_{i-1}$;
- 到达 i 后继续处理 $(i, r]$ ，代价 $F(i, r, h)$ 。

$$F(l, r, h) = \min \left(F(l, r, h - 1), \min_{i=l+1}^r \{ F(l, i - 1, h - 1) + S_r - S_{i-1} + F(i, r, h) \} \right).$$

跳跃表

子任务五 (45 分, 无附加限制): 当 $r - l > h > 1$ 时使用

$$F(l, r, h) = \min_{i=l+h}^r \{F(l, i-1, h-1) + S_r - S_{i-1} + F(i, r, h)\}.$$

记忆化搜索或按区间长度递推均可。状态数为 $O(KN^2)$, 每个状态至多枚举 $O(N)$ 个首节点, 总时间复杂度上界为 $O(KN^3)$, 空间复杂度为 $O(KN^2)$ 。
答案为 $F(0, N, K)$ 。总代价可能超过 32 位整数范围, 必须使用 64 位整数。

位置 i 可以选择前 a_i 个大写字母之一。统计不同字符串 T , 使

$$\text{LCS}(T, S) \geq n - 2.$$

$$n \leq 3 \times 10^5, \quad a_i \leq 26.$$

答案对 998244353 取模。

幸运评级

子任务一 (20 分, $n \leq 4$): 深度优先枚举每个位置允许的字母。得到完整的 T 后, 用普通二维 DP 计算 $\text{LCS}(T, S)$ 。
枚举数至多为

$$26^4 = 456976.$$

时间复杂度为 $O(\prod_i a_i \cdot n^2)$, 空间复杂度为 $O(n^2)$ 。
当 $n \leq 2$ 时, $n - 2 \leq 0$, 所以每个满足逐位字母限制的字符串都合法。

设一个长度至少为 $n - 2$ 的公共子序列把 T_i 与 S_j 匹配。它在 $T_1 \dots T_i$ 、 $S_1 \dots S_j$ 中已经匹配了同样多的字符；两边各自没有匹配的字符都不超过 2，所以

$$|i - j| \leq 2.$$

也就是说，若 T_i 参与最终所需的公共子序列，它只可能与

$$S_{i-2}, S_{i-1}, S_i, S_{i+1}, S_{i+2}$$

之一匹配。偏离主对角线超过 2 的匹配不可能出现在合法方案中，因此只需维护 LCS 表中主对角线附近的五格。

观察

对于可能合法的 T , 存在一条始终位于 $|i - j| \leq 2$ 带内、最终匹配至少 $n - 2$ 个字符的 LCS 路径。令

$$f[i][j]$$

表示只使用带内转移时, $T_1 \dots T_i$ 与 $S_1 \dots S_j$ 的最大匹配数。带内仍使用普通 LCS 转移:

$$f[i][j] = \max\{f[i-1][j], f[i][j-1], f[i-1][j-1] + [T_i = S_j]\}.$$

处理第 i 位时, 只计算 $j \in [i-2, i+2]$ 。上一行需要的五格是 $f[i-1][i-3], \dots, f[i-1][i+1]$; 按 j 从小到大即可算出新五格。越出宽度 2 的转移直接丢弃, 因为它不可能属于长度 $n - 2$ 的公共子序列。

同一行相邻两个 LCS 值之差只能是 0 或 1。令

$$\Delta_t = f[i][i+t] - f[i][i+t-1], \quad -1 \leq t \leq 2,$$

则四个 Δ_t 可以用一个四位二进制数保存。

注意到每增加一位, LCS 最多增加 1, 这个差不可能再减小, 可以直接丢弃。

于是状态只有 $2^4 \times 3 = 48$ 种 (这里 3 是因为 $f[i][j-2]$ 的取值只有 $\min(i, j-2), \min(i, j-2) - 1, \min(i, j-2) - 2$ 三种)。

幸运评级

子任务二 (60 分, $n \leq 10^4$): 令 $dp[mask][d]$ 表示处理完当前前缀后, 四个差分为 $mask$ 、主对角线差为 d 的评级串数量。

加入第 i 个字符时, 枚举

$$c \in \{A, B, \dots, A + a_i - 1\},$$

先由旧状态还原上一行五格, 再按 LCS 递推式从左到右计算新五格, 最后重新编码成 $(mask, d)$ 。初始状态为 $(0, 0)$, 使用滚动数组转移。

处理完 n 位后, 保留下来的状态都满足 $n - f[n][n] \leq 2$, 将它们的计数求和即为答案。

时间复杂度为 $O(48 \cdot 26 \cdot n)$ 。

子任务三 (20 分, 无附加限制): 注意到不需要枚举所有 26 种字符, 因为本质上如果一个字符与 $S_{i-2}, S_{i-1}, S_i, S_{i+1}, S_{i+2}$ 的大小关系相同, 那么 DP 过程不会变。每个状态只需真正计算一次; 同类的 w 个字母可以一次加入 w 倍贡献。考虑到 $S_{i-2}, \dots, S_{i+2}$ 最多只有五种字母, 所以最多有六种字母。时间复杂度为

$$O(48 \cdot 6 \cdot n)$$

打麻将

有 N 种无限张牌。刻子是三张同号牌，顺子是三个连续编号。给定初始手牌后，每次等概率摸一种牌；求第一次能够组成至少 K 个互不复用面子所需摸牌数的期望。

$$N, M \leq 100, \quad K \leq 20.$$

答案在模 998244353 下计算。

设 g_j 为有序摸 j 张牌后仍未和牌的摸牌序列数。每个长度为 j 的序列概率都是 N^{-j} 。由尾和公式，

$$\mathbb{E}[T] = \sum_{j \geq 0} \Pr(T > j) = \sum_{j \geq 0} \frac{g_j}{N^j}.$$

只需按摸牌张数统计尚未和牌的有序序列。若初始已经和牌，则 $g_0 = 0$ ，答案自然为 0。

观察

任取未和牌手牌中的最大面子集合，至多有 $K-1$ 个面子。删去这 $3(K-1)$ 张牌后，余下手牌不含面子：

- 每种牌最多剩两张，否则还能取刻子；
- 任意三个连续编号中至少有一种为零，否则还能取顺子。

把编号每三个分成一组，无面子手牌至多有

$$B(N) = 4 \left\lfloor \frac{N}{3} \right\rfloor + 2(N \bmod 3)$$

张。故额外摸牌只需统计到 $L = \max(0, B(N) + 3(K-1) - M)$ ；超过 L 后必然已经和牌。

打麻将

子任务一 (10 分, $N \leq 2$): 没有顺子, 只需统计刻子。

按牌种处理。状态记录已经摸了 j 张、组成了 $c < K$ 个刻子。当前牌种初始有 b 张, 又摸到 y 张时, 新增

$$\left\lfloor \frac{b+y}{3} \right\rfloor$$

个刻子。把这 y 张同类牌插入此前 j 张有序序列的方案数为 $\binom{j+y}{y}$ 。

按照当前部分分程序, 时间复杂度为 $O(NKL^2)$, 空间复杂度为 $O(KL)$ 。

打麻将

子任务二 (20 分, $N \leq 3$): 固定总摸牌数 j , 枚举

$$x_1 + x_2 + x_3 = j.$$

这一计数向量对应的有序摸牌序列数为

$$\frac{j!}{x_1!x_2!x_3!}.$$

加入初始手牌后, 再枚举使用多少个顺子 $(1, 2, 3)$; 剩余每种牌尽量组成刻子, 即可判断是否仍未和牌。

$L \leq 61$, 直接实现的时间复杂度上界为 $O(L^4)$, 足以通过该子任务。

打麻将

子任务三 (20 分, $K = 1$): 未和牌等价于

- 每种牌至多两张;
- 任意三个连续编号不全都出现。

按牌号扫描, 用两位掩码记录前两种牌是否出现。当前牌额外摸 y 张时, 考虑组合数的分母, 因此需要乘 $1/y!$; 如果三位连续为 1, 或当前总数超过 2, 则转移非法。处理完所有牌种后, 长度 j 的无序计数乘 $j!$, 恢复有序摸牌序列数 g_j 。状态数为 4, 时间复杂度为 $O(NL)$, 空间复杂度为 $O(L)$ 。

观察

满分需要在扫描牌号时求已经处理部分最多能组成多少面子。处理当前牌号 i 之前，用 3×3 表 $F[a][b]$ 表示：

- a ：已经有 a 个 $(i-2, i-1)$ 两连张，等待牌 i ；
- b ：保留了 b 张牌 $i-1$ ，准备与 $i, i+1$ 组成顺子；
- $F[a][b]$ ：在这种跨边界结构下已经完成的最多面子数。

一张自动机状态是 a, b 分别从 0 到 2 枚举的 3×3 九元表。不可达表项记为 $-\infty$ 。同时已经赢也是一种状态。

观察

a, b 只需取 $0, 1, 2$: 三个相同的未完成结构可以改用刻子处理, 面子数不会变差。

设当前牌号共有 x 张。从旧表项 (a, b) 出发:

- ① 先用 $c = \min(a, x)$ 张补完旧两连张。少补一个会损失一个已经只差一张的面子, 而留下的当前牌至多只能参与一个别的面子, 所以补到不能补为止不劣;
- ② 剩余 $r = x - c$ 张。枚举 $a' \leq \min(b, r)$, 把 a' 张与旧的牌 $i - 1$ 配成新两连张;
- ③ 枚举 $b' \leq \min(2, r - a')$, 保留 b' 张当前牌; 其余牌组成刻子。

$$F'[a'][b'] = \max \left(F'[a'] [b'], F[a] [b] + c + \left\lfloor \frac{r - a' - b'}{3} \right\rfloor \right).$$

打麻将

若新表任一项达到 K ，统一转入和牌吸收态。从初始表 $F[0][0] = 0$ 、其余为 $-\infty$ 出发，对所有可能的当前牌数 x 做 BFS，枚举全部可达九元表，并对每个 s 预处理会到达的状态。

九元表已经对同一边界结构取最大面子数：以后只关心还能否达到 K ，较小值不会优于较大值。因此相同九元表的历史可以合并，且这张转移表能在所有牌号上复用。

打麻将

令 $dp[i][j][s]$ 表示已经处理到第 i 种牌（这一维可以压掉），共额外摸了 j 张，自动机状态为 s 的有序摸牌序列数。

处理下一牌种时，若初始有 b 张、额外摸到 y 张，则转到

$$dp[i+1][j+b+y][to[s][b+y]].$$

把 y 张新类型牌插入此前 j 张序列的方案数为

$$\binom{j+y}{y}.$$

逐牌种乘这些组合数，算出真实的方案数，每个有序摸牌序列被计算一次。

打麻将

子任务四 (50 分, 无附加限制): 处理完全部牌种后, 对每个状态, 计算 DP 值。若可达自动机状态数为 S , 时间复杂度上界为 $O(NSL^2)$ 。枚举某牌种的新增张数时, 一旦转入吸收态即可停止继续增加。
所有计数、组合数和逆元均在模 998244353 下计算。

树上守望

带权树上选择若干付费守望点。每个节点 u 必须被某个已选点 x 覆盖：

$$\text{dist}(u, x) \leq d_u.$$

求最小总费用。

$$n \leq 1000.$$

树上守望

子任务一 (15 分, $n \leq 20$): 枚举非空守望点集合 s 。
处理出任意点 i 到任意点集的最小距离。

$$dp[i][s] = \min(dp[i][s \setminus \text{lowbit}(s)], dp[i][\text{lowbit}(s)]);$$

时间复杂度为 $O(2^n n)$ 。

树上守望

子任务二 (15 分, 树是一条链):

考虑选了两个关键点 l, r , 我们可以 $O(r - l)$ 去 check 这两个点能不能覆盖这个区间。对于一个 l 而言, 合法的 r 一定是一个 $[l, R]$, 因此可以二分出 r 的最大值 R 。

令 $dp[i]$ 表示前缀 $1, 2, \dots, i$ 已覆盖、守望点 i 已选的最小代价, 转移为:

$$dp[i] = \min_{R(j) \geq i} (dp[j]) + c_i$$

时间复杂度 $O(n^2 \log n)$ 。

树上守望

子任务三 (15 分, 除节点 1 外均与 1 相连): 设叶子 i 的边长为 w_i 。

若选择中心 1, 满足 $w_i \leq d_i$ 的叶子已被覆盖; 其余叶子只能选择自己。

若不选择中心, 枚举离中心最近的已选叶子 p 。必须有 $w_p \leq d_1$, 否则中心未被覆盖。

对另一叶子 $i \neq p$, 任意其他已选叶子 q 都有 $w_q \geq w_p$, 因此

$$\text{dist}(i, q) = w_i + w_q \geq w_i + w_p.$$

若 $w_i + w_p > d_i$, 叶子 i 只能选择自己; 否则 p 已能覆盖它。叶子 p 的费用已经在枚举它时计入。枚举 p 后扫描全部其他叶子, 时间复杂度为 $O(n^2)$ 。

子任务四 (15 分, 所有 $d_i = 1$ 且边权 $w = 1$): 问题变成带点权树上的最小权支配集。
对每个节点维护三种状态:

- ① A_u : 选择了 u ;
- ② B_u : 未选择 u , 但已被某个儿子覆盖;
- ③ C_u : 未选择 u 且子树内尚未覆盖它, 要求父亲被选中。

A_u 的儿子可取任意状态; C_u 的所有儿子必须取 B ; B_u 的儿子只能取 A/B , 并且至少一个取 A 。合并时维护尚无儿子选中与已经有儿子选中的状态即可。
根不能处于 C , 答案为 $\min(A_1, B_1)$ 。时间复杂度为 $O(n)$ 。

在第五个子任务中，边权全为 1、每个点作为守望点的费用相等。把树根定为 1，定义

$$\text{key}(u) = \text{depth}(u) - d_u.$$

按 key 从大到小处理。若 u 尚未覆盖，就选择从 u 向上走 d_u 步的祖先；若走不到这么多步则选择根。

理由：考虑第一个被选择的 u 与其 d_u 步的祖先 v ，为了覆盖到 u ，只能从 v 的子树里选守望点，而此时 v 能覆盖的范围是最多的。然后剩下的问题变成一个子问题，可以归纳。

子任务五（20 分，单位边权且所有 c_i 相等）：使用上一页的贪心。
每次选择守望点后，标记它能够覆盖的所有节点。直接预处理全点对距离，总复杂度为 $O(n^2)$ 。
因为所有费用相同，最小费用等价于最少守望点数；若费用不等，上一页的交换不再保证代价不增加。

观察

固定一份可行方案。对每个节点 u ，指定距离最近的已选守望点 $p(u)$ 。设 v 是 u 的儿子， $p(u) = x$ 。

若 $p(v) \neq x$ ，则 $p(v)$ 一定位于 v 子树内：

- 若 x 在 v 子树内，任何比 x 更靠近 v 的点也会更靠近 u ，与 $x = p(u)$ 矛盾；
- 若 x 在子树外，所有子树外候选从 u 到 v 都增加同一条边长，最近的仍是 x 。

所以每个儿子或者继续共享 x ，或者由子树内部独立负责。

树上守望

预处理全部点对距离。定义 $dp[u][x]$ 为一份 u 的守望点为 x 的子树内最小代价：

- ① 要求守望点集能覆盖 u 的整棵子树；
- ② 集合包含指定点 x ，且 $\text{dist}(u, x) \leq d_u$ ；

这里不要求合并完成后 x 仍是实际离 u 最近的守望点。
若距离条件不满足，状态不可达；否则初始化 $dp[u][x] = c_x$ 。
令 $f[u] = \min_x dp[u][x]$ 。

合并儿子 v 时有两种选择：

- v 子树独立负责，代价为 $f[v]$ ；
- 父子共享 x ，代价为 $dp[v][x]$ ，但双方都计算了 c_x ，需要减去一次。

因此

$$dp[u][x] += \min(f[v], dp[v][x] - c_x).$$

所有儿子合并完后重新计算 $f[u]$ 。答案为 $f[1]$ 。

子任务六 (20 分, 无附加限制): 从每个源点遍历树一次, $O(n^2)$ 预处理全部加权距离; 再按后序计算 $dp[u][x]$, 每条父子边对所有 x 扫描一次。

- 时间复杂度为 $O(n^2)$;
- 空间复杂度为 $O(n^2)$ 。

距离与答案使用 64 位整数, 避免 INF 溢出。多组数据之间要完整清空状态。